EGCO 425 – Chapter 11

Clustering Methods

- Hierarchical Clustering
- DBSCAN

References

[1] Tan et al., Introduction to Data Mining (Chapter 8)

[2] Kotu & Deshpande, Data Science: Concepts and Practice (Chapter 7)

[3] RapidMiner Doc, https://docs.rapidminer.com/latest/studio/operators/

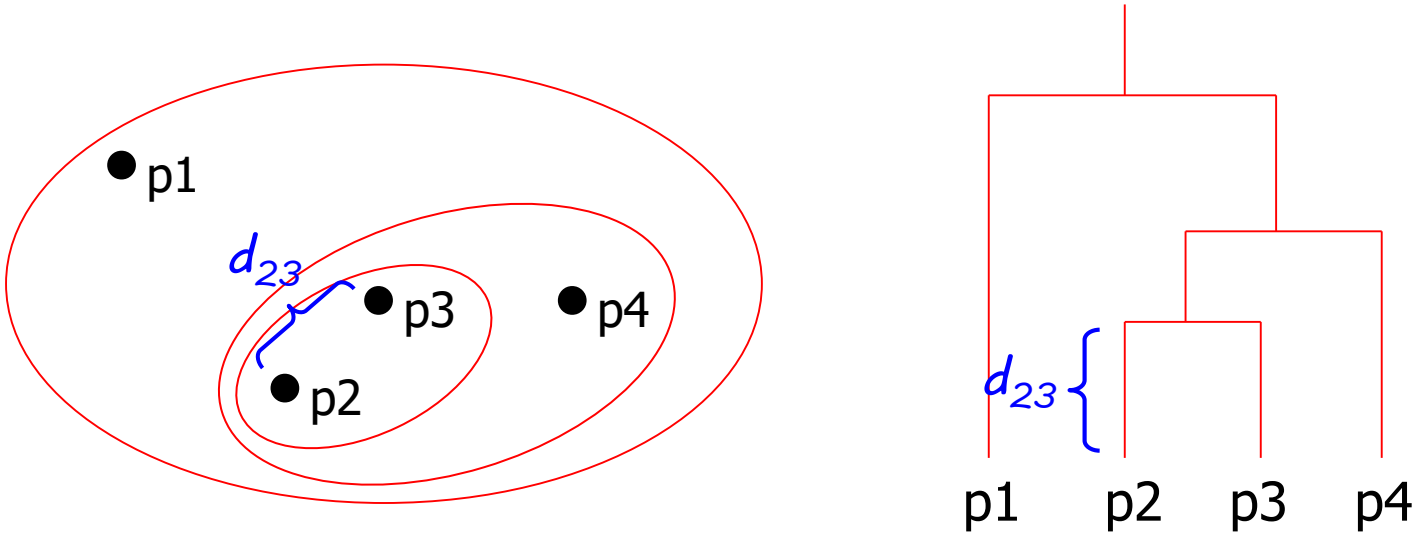
R. Marukatat

Outline

- Hierarchical agglomerative clustering (HAC)
  - Dendrogram
  - Cophenetic distance, cophenetic correlation
  - Linkage type ➔ single link, complete link, group average
- DBSCAN
  - MinPts, epsilon
  - Data point classification ➔ core, border, noise
- Visualization
  - https://educlust.dbvis.de/

Hierarchical Clustering

- Output nested clusters, which can be represented as dendrogram
  - Height of dendrogram = cophenetic distance between 2 clusters
  - No need to specify K ➔ any required #clusters can be obtained by cutting a dendrogram

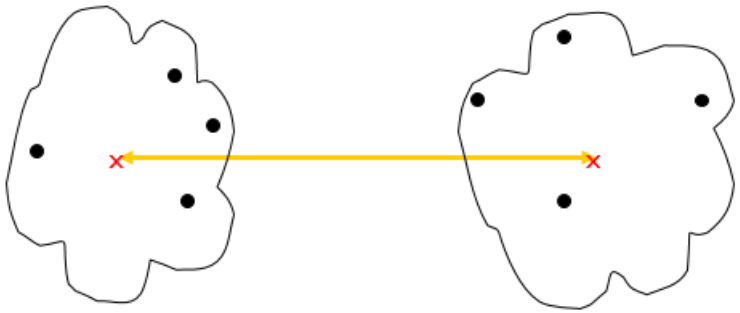


Clustering Approaches

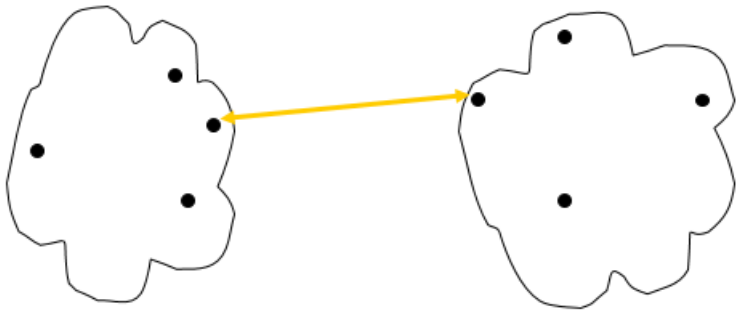
- Agglomerative approach
  - Start with points as individual clusters
  - Repeatedly merge closest clusters (according to cophenetic distance) until only one outermost cluster is obtained
  - Easier to implement & more popular than divisive approach
- Divisive approach
  - Start with one, all-inclusive cluster
  - Repeatedly split clusters until each cluster contains a point

Linkage Types

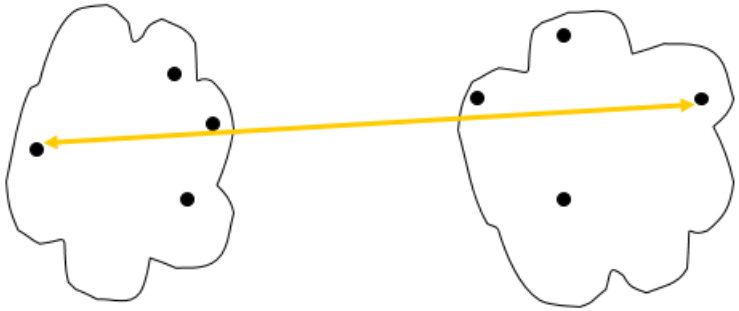
- When considering distance between 2 clusters
  - Each cluster contains many points. There are many possible distances between 2 clusters
  - Cophenetic distance = representative distance between a point in one cluster to a point in another cluster
- How to calculate cophenetic distance ?
  - Distance between centroids
    - Easy to calculate but tend to give worse results than others
  - Single link = shortest distance between points from different clusters
  - Complete link = longest distance between points from different clusters
  - Group average = average of all possible distances between clusters
  - Different linkage types give different clustering results



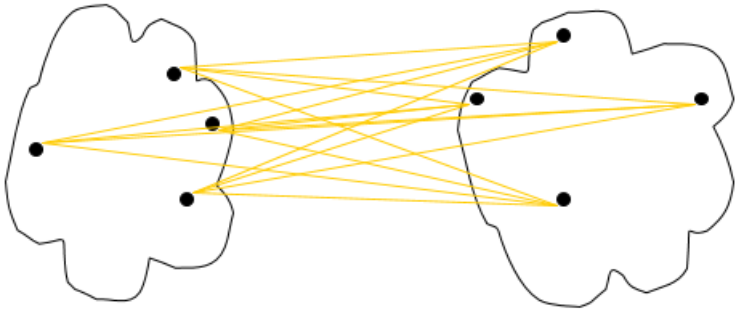
Distance between centroids



Single link (nearest neighbor)

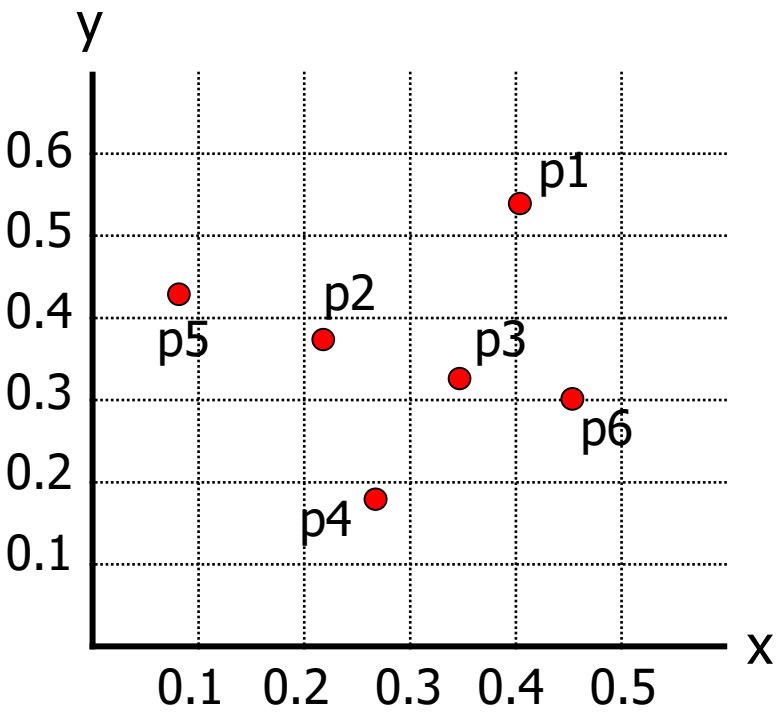


Complete link (farthest neighbor)



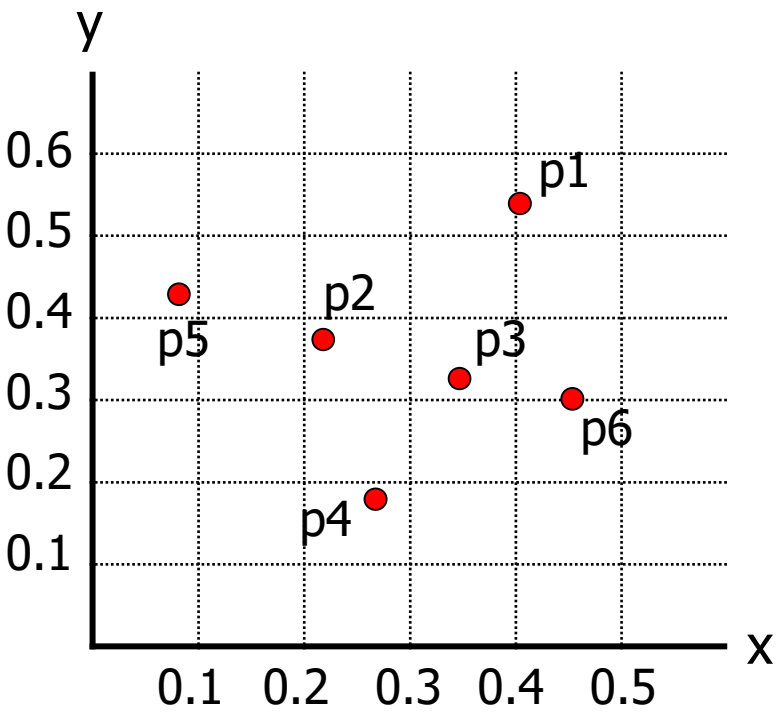
Group average

Hierarchical Agglomerative Clustering (HAC)



Coordinate

	x	y
p1	0.40	0.53
p2	0.22	0.38
p3	0.35	0.32
p4	0.26	0.19
p5	0.08	0.41
p6	0.45	0.30



Euclidean Distance

	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2		0.00	0.15	0.20	0.14	0.25
p3			0.00	0.15	0.28	**0.11**
p4				0.00	0.29	0.22
p5					0.00	0.39
p6						0.00

Single link

	p1	p2	p3+p6	p4	p5
p1	0.00	0.24	0.22	0.37	0.34
p2		0.00	0.15	0.20	**0.14**
p3+p6			0.00	0.15	0.28
p4				0.00	0.29
p5					0.00

d(1, 3+6) = min{ d(1,3), d(1,6) }
= min{ 0.22, 0.23 } = 0.22

d(2, 3+6) = min{ d(2,3), d(2,6) }
= min{ 0.15, 0.25 } = 0.15

d(3+6, 4) = min{ d(3,4), d(6,4) }
= min{ 0.15, 0.22 } = 0.15

d(3+6, 5) = min{ d(3,5), d(6,5) }
= min{ 0.28, 0.39 } = 0.28

After merging p3+p6

- Update distance matrix by calculating cophenetic distance between p3+p6 and other clusters (or individual points)
- From the new distance matrix ➔ merge p2 and p5

Single link

	p1	p2+p5	p3+p6	p4
p1	0.00	0.24	0.22	0.37
p2+p5		0.00	**0.15**	0.20
p3+p6			0.00	0.15
p4				0.00

d(1, 2+5) = min{ d(1,2), d(1,5) }
= min{ 0.24, 0.34 } = 0.24

d(2+5, 3+6) = min{ d(2,3+6), d(5,3+6) }
= min{ 0.15, 0.28 } = 0.15

d(2+5, 4) = min{ d(2,4), d(5,4) }
= min{ 0.20, 0.29 } = 0.20

Cophenetic distance
cd(2,3) = cd(2,6) = cd(5,3) = cd(5,6) = 0.15

After merging p2+p5

- Update distance matrix by calculating cophenetic distance between p2+p5 and other clusters (or individual points)
- From the new distance matrix ➔ merge {p2+p5} and {p3+p6}

Complete link

	p1	p2	p3+p6	p4	p5
p1	0.00	0.24	0.23	0.37	0.34
p2		0.00	0.25	0.20	**0.14**
p3+p6			0.00	0.22	0.39
p4				0.00	0.29
p5					0.00

d(1, 3+6) = max{ d(1,3), d(1,6) }
= max{ 0.22, 0.23 } = 0.23

d(2, 3+6) = max{ d(2,3), d(2,6) }
= max{ 0.15, 0.25 } = 0.25

d(3+6, 4) = max{ d(3,4), d(6,4) }
= max{ 0.15, 0.22 } = 0.22

d(3+6, 5) = max{ d(3,5), d(6,5) }
= max{ 0.28, 0.39 } = 0.39

After merging p3+p6

- Update distance matrix by calculating cophenetic distance between p3+p6 and other clusters (or individual points)
- From the new distance matrix ➔ merge p2 and p5

Complete link

	p1	p2+p5	p3+p6	p4
p1	0.00	0.34	0.23	0.37
p2+p5		0.00	0.39	0.29
p3+p6			0.00	**0.22**
p4				0.00

d(1, 2+5) = max{ d(1,2), d(1,5) }
= max{ 0.24, 0.34 } = 0.34

d(2+5, 3+6) = max{ d(2,3+6), d(5,3+6) }
= max{ 0.25, 0.39 } = 0.39

d(2+5, 4) = max{ d(2,4), d(5,4) }
= max{ 0.20, 0.29 } = 0.29

After merging p2+p5

- Update distance matrix by calculating cophenetic distance between p2+p5 and other clusters (or individual points)
- From the new distance matrix ➔ merge {p3+p6} and p4

Group average

	p1	p2	p3+p6	p4	p5
p1	0.00	0.24	0.225	0.37	0.34
p2		0.00	0.20	0.20	**0.14**
p3+p6			0.00	0.185	0.335
p4				0.00	0.29
p5					0.00

d(1, 3+6) = avg{ d(1,3), d(1,6) }
= avg{ 0.22, 0.23 } = 0.225

d(2, 3+6) = avg{ d(2,3), d(2,6) }
= avg{ 0.15, 0.25 } = 0.20

d(3+6, 4) = avg{ d(3,4), d(6,4) }
= avg{ 0.15, 0.22 } = 0.185

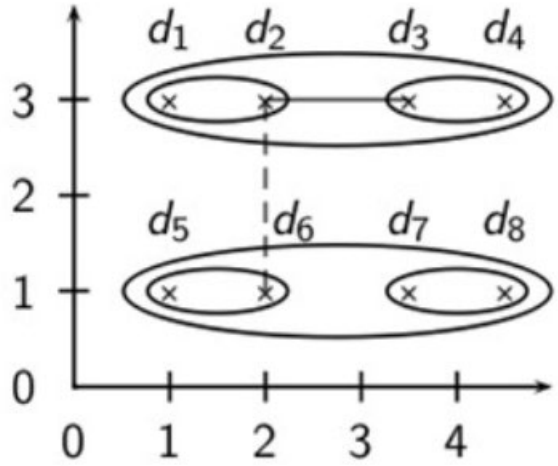
d(3+6, 5) = avg{ d(3,5), d(6,5) }
= avg{ 0.28, 0.39 } = 0.335

After merging p3+p6

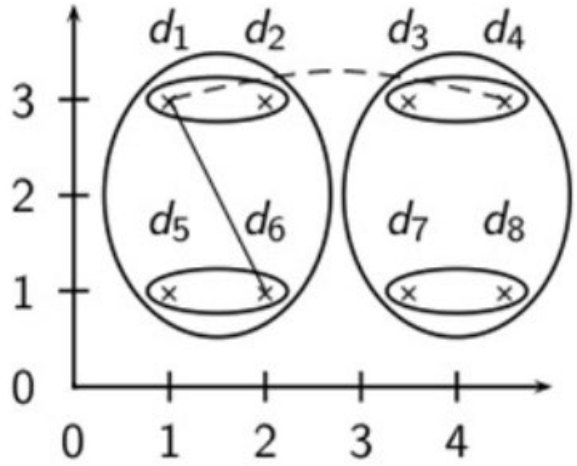
- Update distance matrix by calculating cophenetic distance between p3+p6 and other clusters (or individual points)
- From the new distance matrix ➔ merge p2 and p5

Single Link vs. Complete Link

- Cluster formation



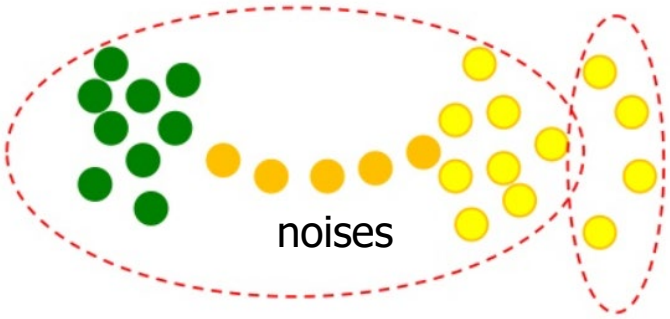
Single link



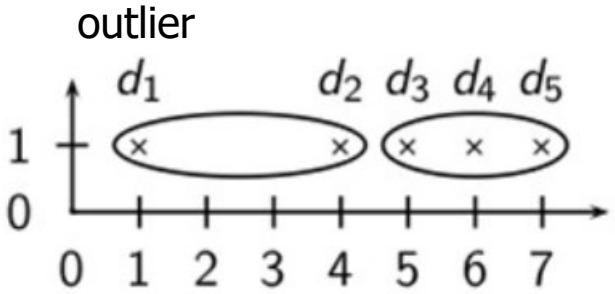
Complete link

- Single link produces clusters of any shape. Cluster sizes can be unequal
- Complete link tends to produce globular clusters of equal sizes (big natural clusters are broken into smaller ones)

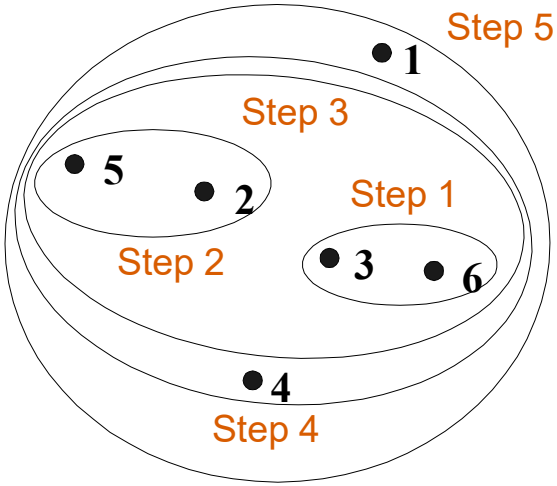
- Single link clusters can be distorted by noises
  - Noises affect points around them and cause chaining effect
  - Green & yellow points are bridged by noises. They are put in 1 cluster
  - Clusters of yellow points are broken



- Complete link clusters can be distorted by outliers
  - Distance to an outlier becomes cophenetic distance, so outliers may be merged early (into a valid cluster)
  - d2 is merged with outlier (i.e. d1) instead of cluster d3+d4+d5

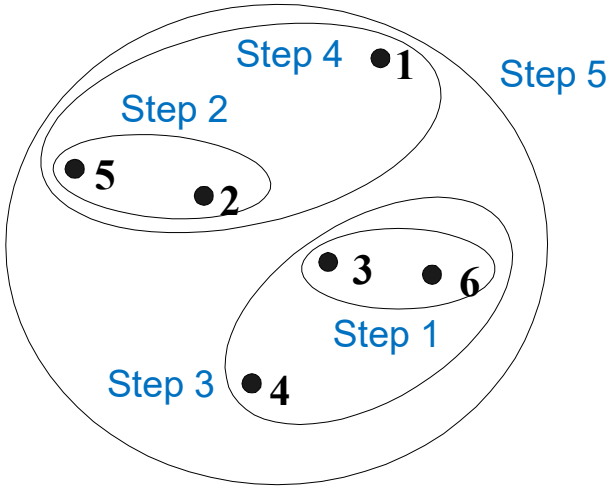


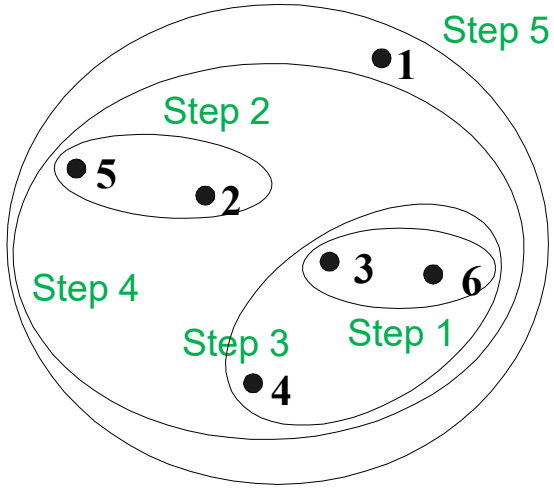
- Group average is a compromise between single and complete link
  - Less susceptible to noises and outliers
  - But also tend to produce globular clusters



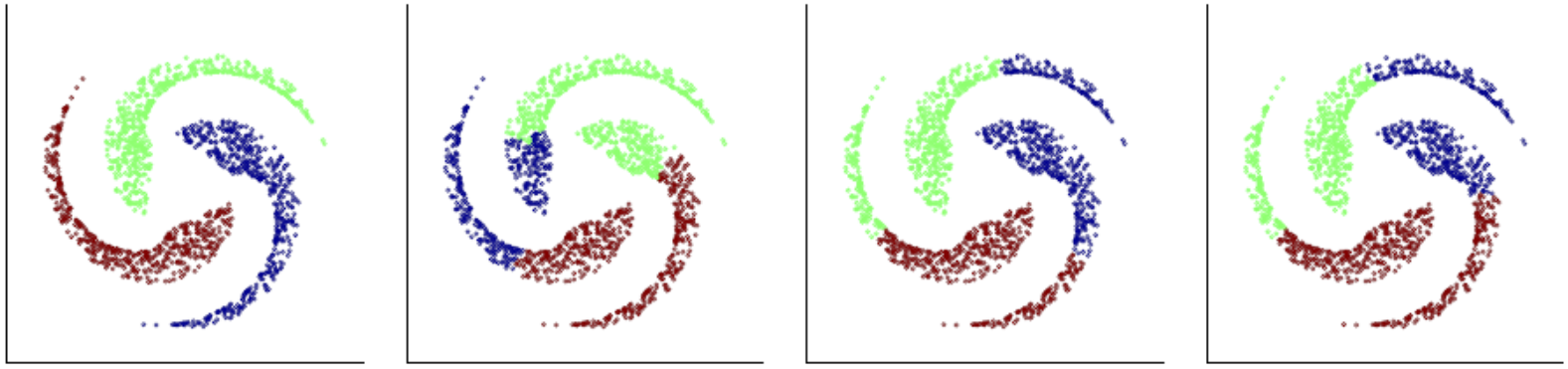
Single link
Cophenetic
correlation = 0.445

Complete link
Cophenetic
correlation = 0.617





Group average
Cophenetic
correlation = 0.655



(a) Single-Linkage (b) Complete-Linkage (c) Average-Linkage (d) Centroid

Single link can produce non-globular clusters
Other links favor globular clusters

Cophenetic Correlation

- Correlation between real distances and cophenetic distances
  - High correlation = cophenetic distances are close to real distances
    = good result
  - But RapidMiner doesn't output this value ➔ use Python

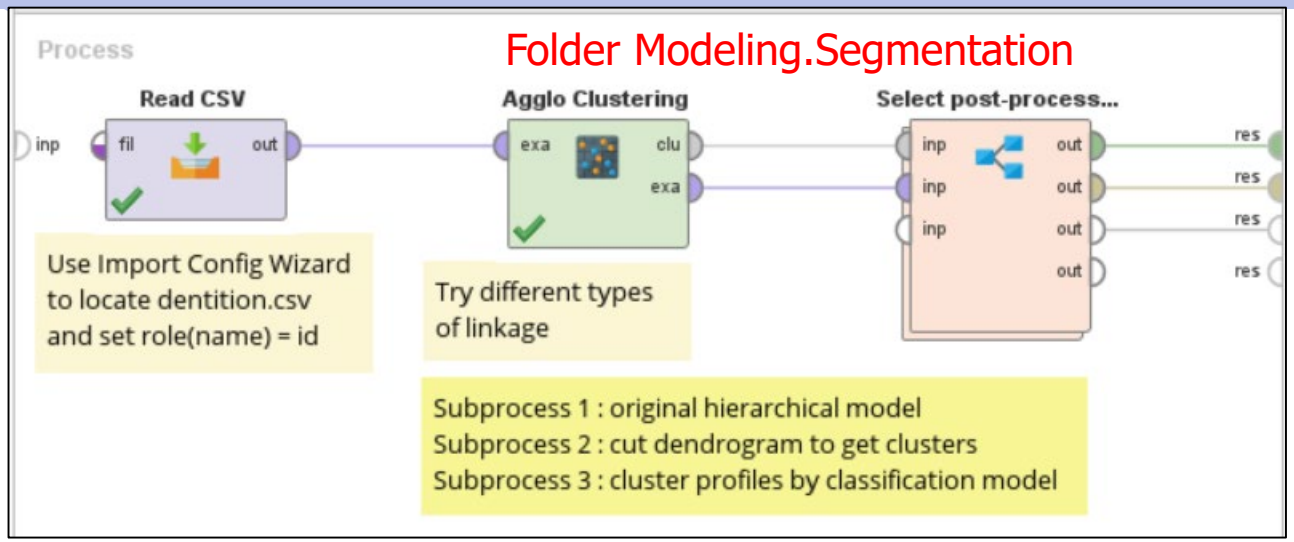
	p1	p2	p3	p4	p5	p6
p1	0.00	0.24	0.22	0.37	0.34	0.23
p2		0.00	0.15	0.20	0.14	0.25
p3			0.00	0.15	0.28	0.11
p4				0.00	0.29	0.22
p5					0.00	0.39
p6						0.00

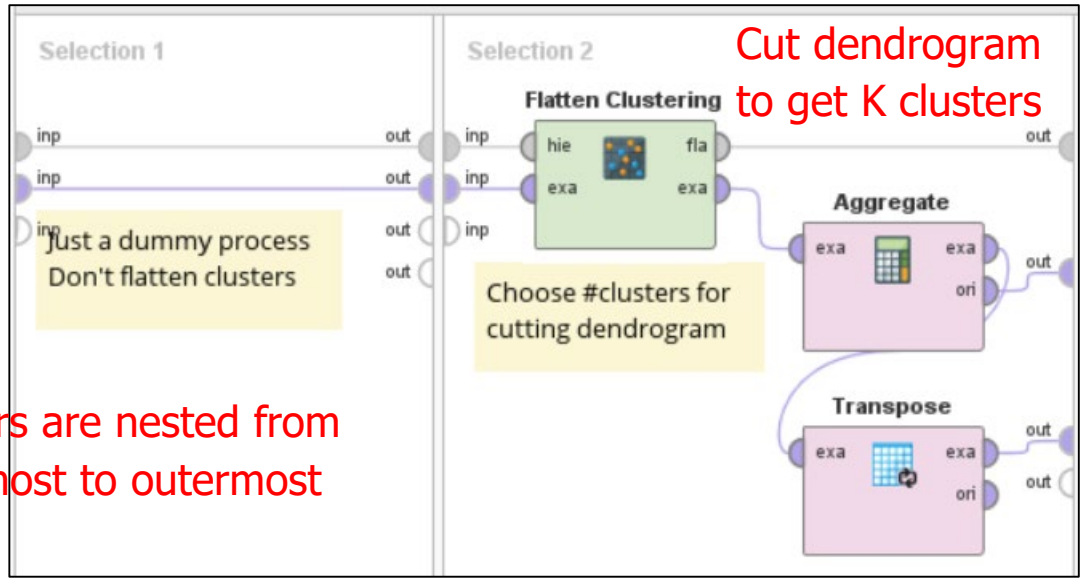
Real distance d_i

	p1	p2	p3	p4	p5	p6
p1	0.00	0.22	0.22	0.22	0.22	0.22
p2		0.00	0.15	0.15	0.14	0.15
p3			0.00	0.15	0.15	0.11
p4				0.00	0.15	0.15
p5					0.00	0.15
p6						0.00

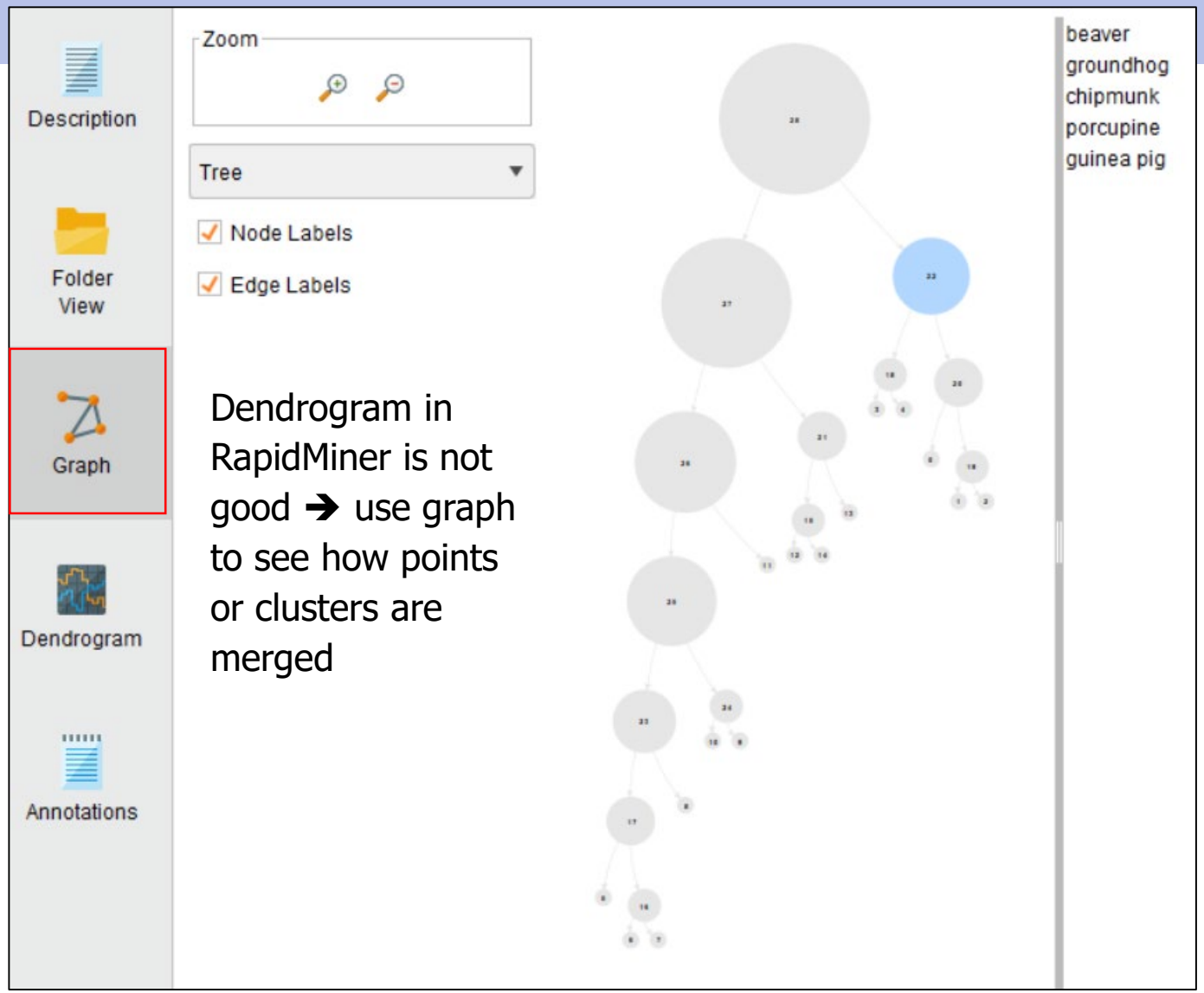
Cophenetic distance cd_i (Single Link)

Example (1) : HAC

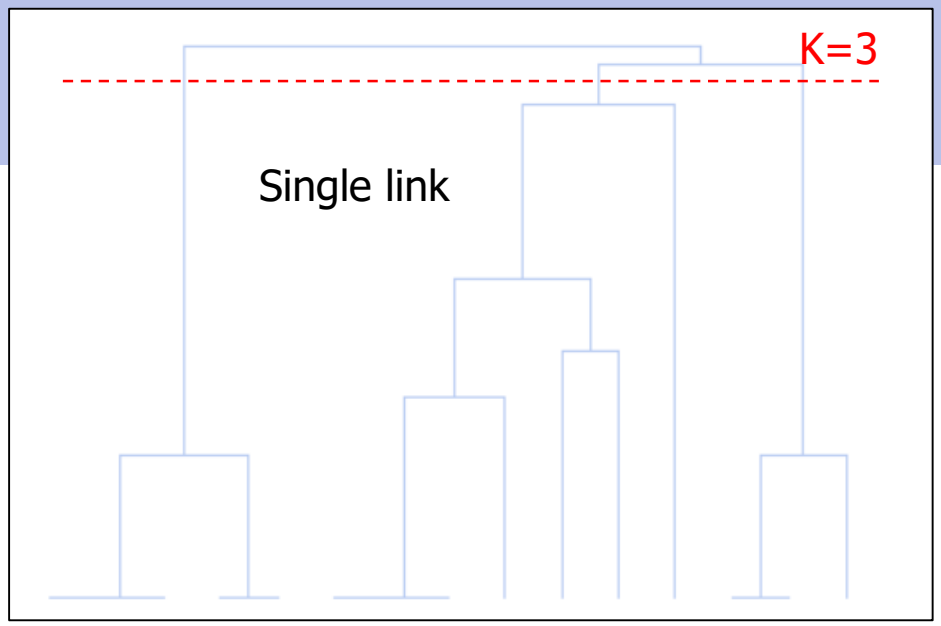


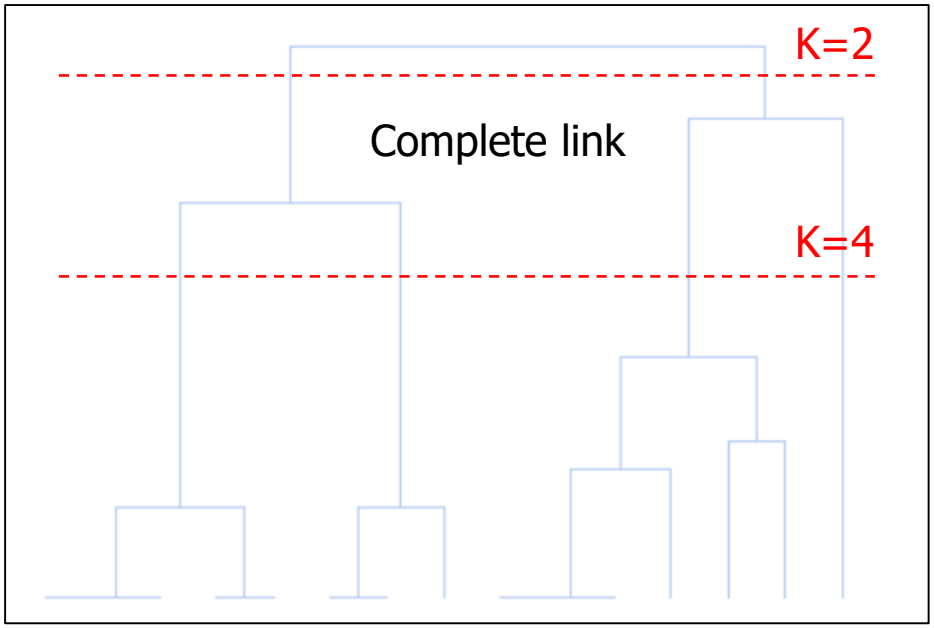


Clusters are nested from innermost to outermost

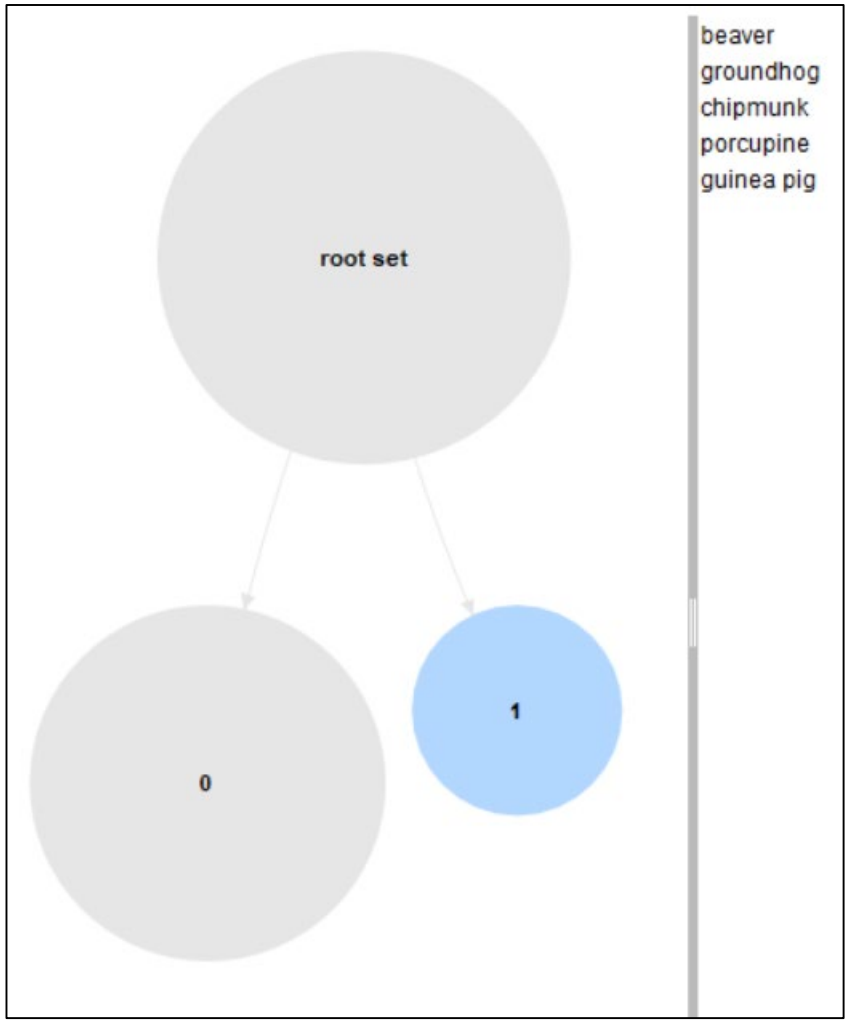


Dendrogram in RapidMiner is not good ➔ use graph to see how points or clusters are merged





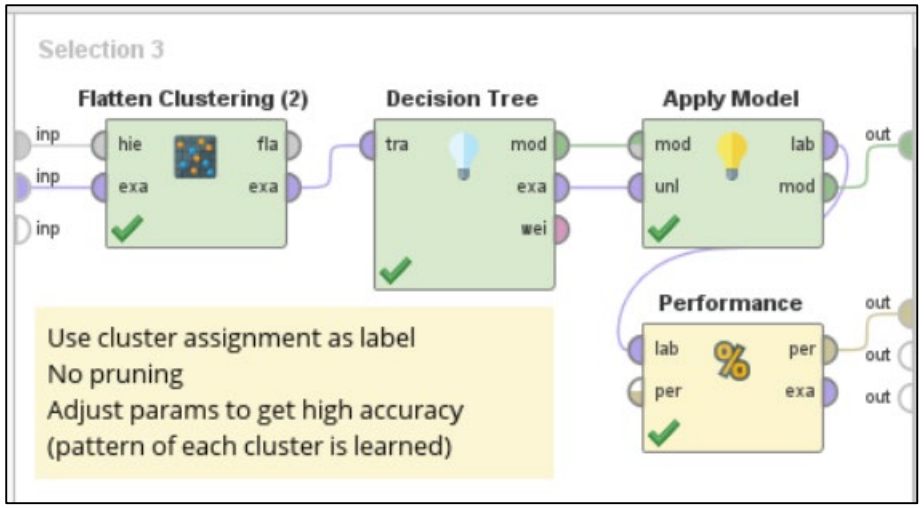
RapidMiner's dendrogram doesn't have record labels. We can only take some hints e.g. where to cut it to get clusters, approx. cluster sizes & distances between them

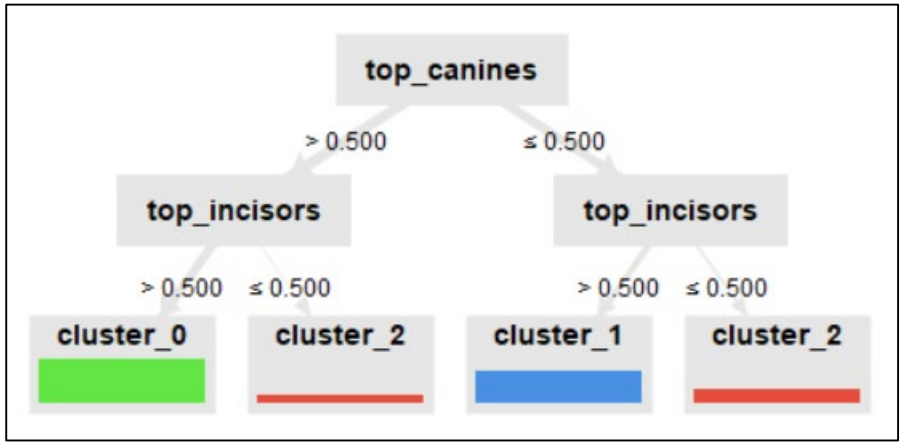


Row No.	id	att_1	att_2
1	count(label)	10.0	5.0
2	average(top_incisors)	1.9	1.0
3	average(bottom_incisors)	2.8	1.0
4	average(top_canines)	0.8	0.0
5	average(bottom_canines)	0.7	0.0
6	average(top_premolars)	3.5	1.6
7	average(bottom_premolars)	3.5	1.0
8	average(top_molars)	2.0	3.0
9	average(bottom_molars)	2.3	3.0
10	label	cluster_0	cluster_1

Flatten single link results
to get 2 clusters (K=2)


Flatten Clustering
top_incisors, bottom_incisors
5
4
3
2
1
0
Members of cluster 1
have similar incisor
patterns
Try K=3 instead of K=2
to further split this cluster
beaver
groundhog
chipmunk
porcupine
guinea pig
wolf
fox
bear
raccoon
sea otter
fur seal
walrus
moose
reindeer
mountain sheep
name
label: cluster_1 (top_incisors)
label: cluster_0 (top_incisors)
label: cluster_1 (bottom_incis...
label: cluster_0 (bottom_incis...



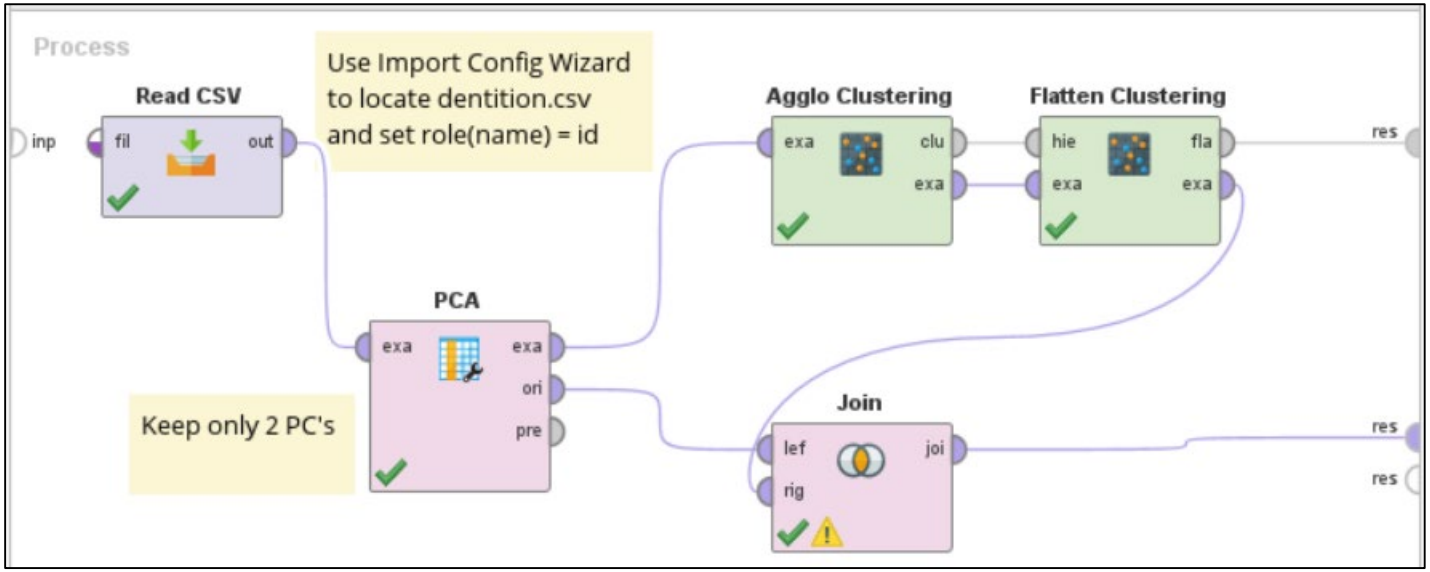


Important attributes that help distinguish between clusters are top canines & top incisors

Use visualization to also check canine patterns

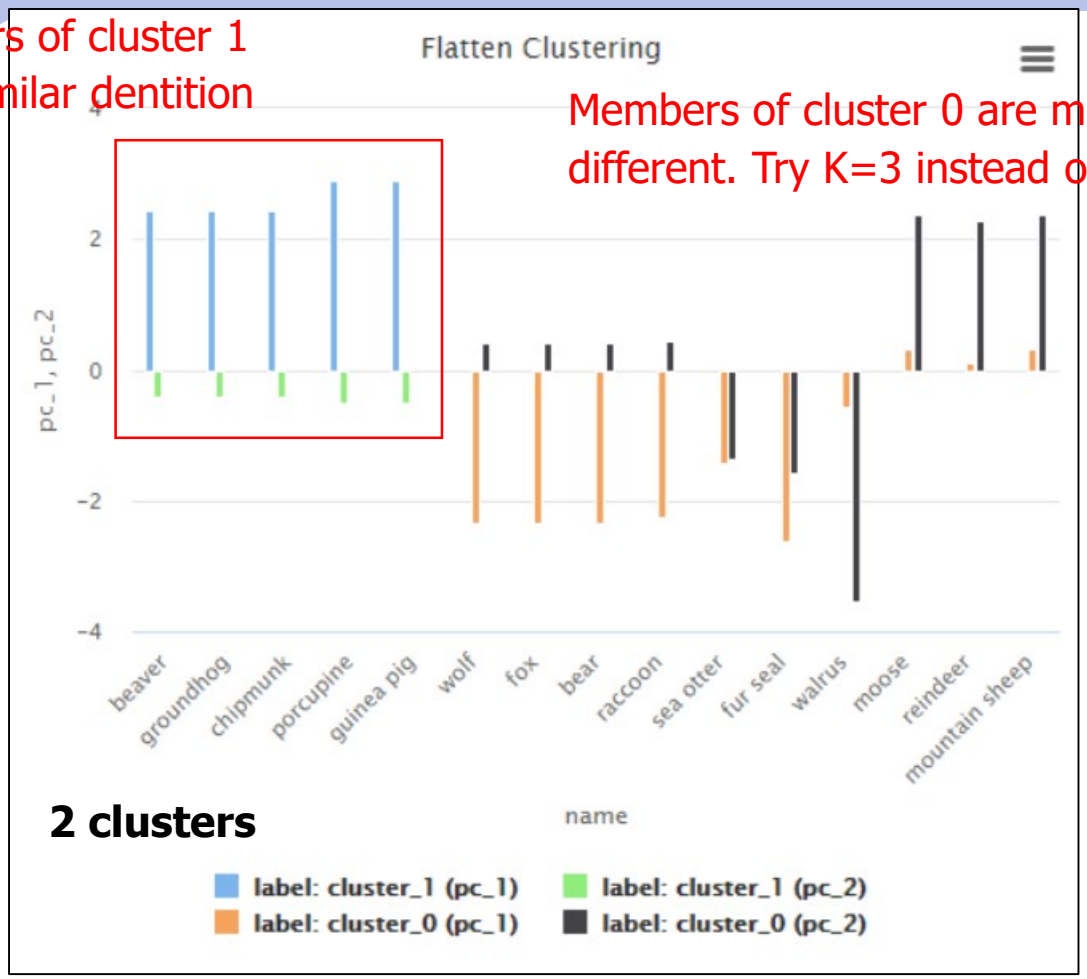
Example (2) : HAC with PCA

Combine 8 dentition attributes into 2 PC's



Then use Aggregate operator or apply classification to analyze cluster profiles

Members of cluster 1 have similar dentition pattern



Same clustering results as Example (1) but PCA makes patterns clearer

Use PCA to check overall patterns & build cluster profiles from original attributes

HAC by Python

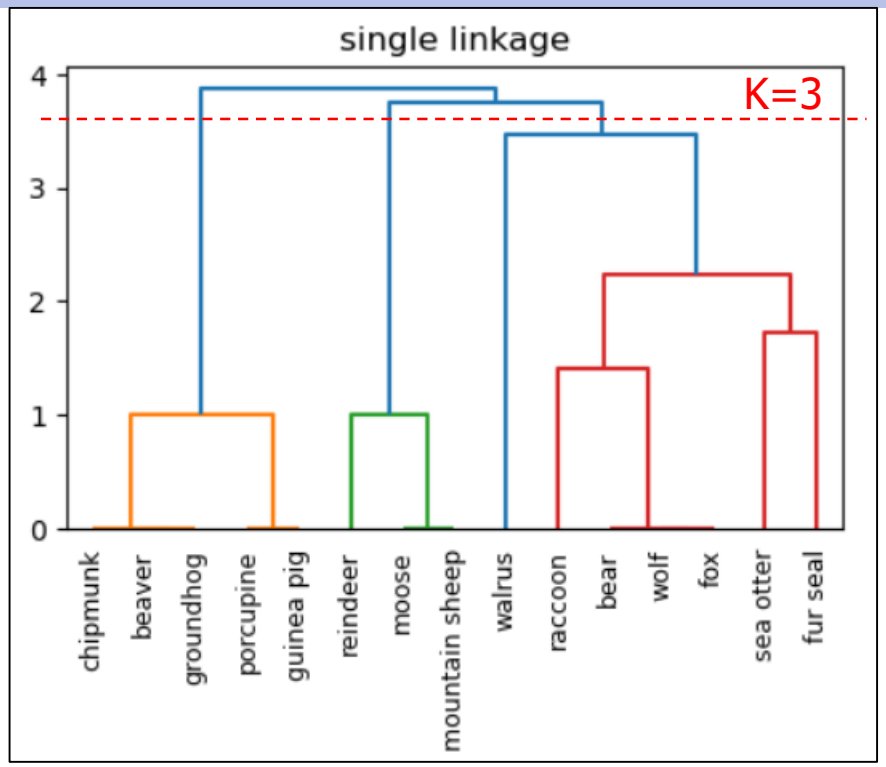
```
DentitionDF.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   name              15 non-null     object
 1   top_incisors      15 non-null     int64
 2   bottom_incisors   15 non-null     int64
 3   top_canines       15 non-null     int64
 4   bottom_canines    15 non-null     int64
 5   top_premolars     15 non-null     int64
 6   bottom_premolars  15 non-null     int64
 7   top_molars        15 non-null     int64
 8   bottom_molars     15 non-null     int64
dtypes: int64(8), object(1)
memory usage: 1.2+ KB
```

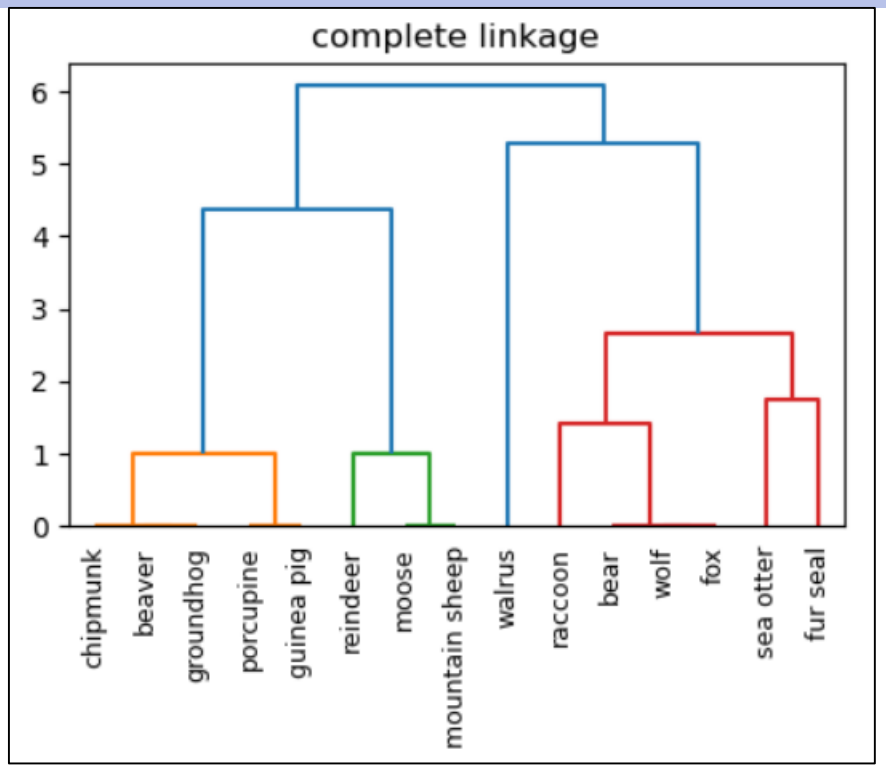
```
### Set name as row index (excluded from distance calculation)

DentitionDF = DentitionDF.set_index('name')
DentitionDF.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 15 entries, beaver to mountain sheep
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   top_incisors      15 non-null     int64
 1   bottom_incisors   15 non-null     int64
 2   top_canines       15 non-null     int64
 3   bottom_canines    15 non-null     int64
 4   top_premolars     15 non-null     int64
 5   bottom_premolars  15 non-null     int64
 6   top_molars        15 non-null     int64
 7   bottom_molars     15 non-null     int64
dtypes: int64(8)
memory usage: 1.1+ KB
```

DataFrame must contain only numeric attributes, with animal name as row index



Cophenetic corr = 0.9376

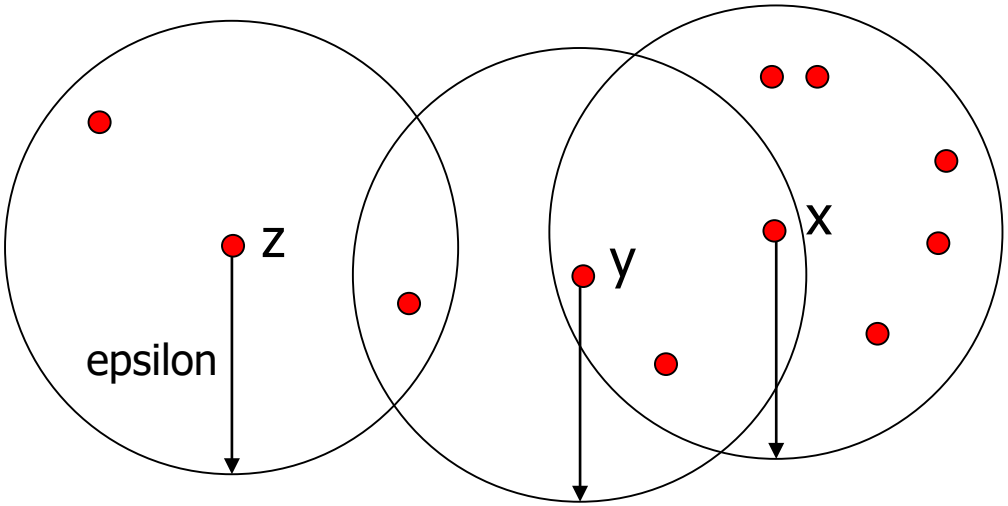


Cophenetic corr = 0.9414

If we cut dendrogram at K=3, compare cluster membership with slide 27

DBSCAN

- Density-based spatial clustering of applications with noise
- Determine whether each data point is in dense region (cluster) or sparse region (noise) by using 2 parameters
  - Radius or epsilon
  - Minimum points (MinPts) threshold. For example, MinPts >= 7



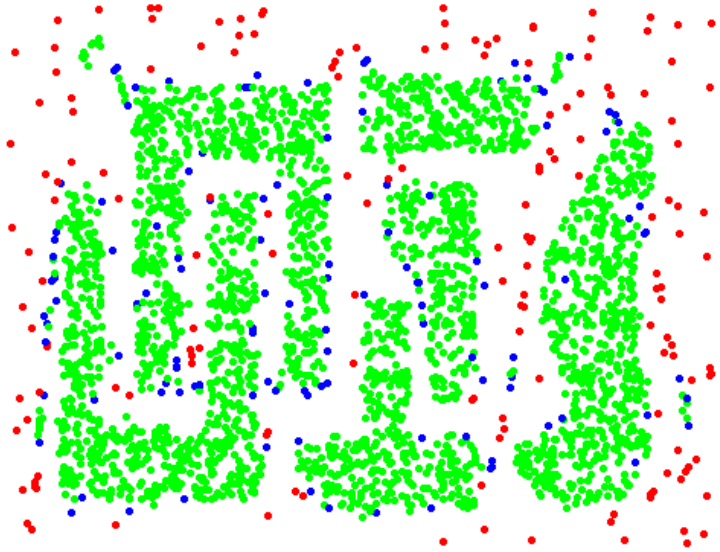
For each point, draw a circle of given radius and count #points inside the radius

Radius(x) = 8 ➔ core point
Radius(y) = 4 ➔ border point
Radius(z) = 3 ➔ noise

- Types of data points
  - Core point ➔ pass MinPts threshold. It is assumed to be at the core of dense region or cluster
  - Border point ➔ fail MinPts threshold but is inside the radius of other core point. It is on the edge of cluster
  - Noise ➔ not core point & not border point. It is in sparse region

- Clustering steps
  1. Label all points as core points, border points, or noises
  2. Remove noises (in RapidMiner, noises are put in separate cluster)
  3. Form clusters by adding links between core points that are within the radius of each other
  4. Assign each border point to cluster whose radius covers it. If a border point is covered by >1 clusters, the decision can be based on
     - Random selection
     - Actual distance — pick nearest cluster
     - Cluster density — pick cluster with highest density

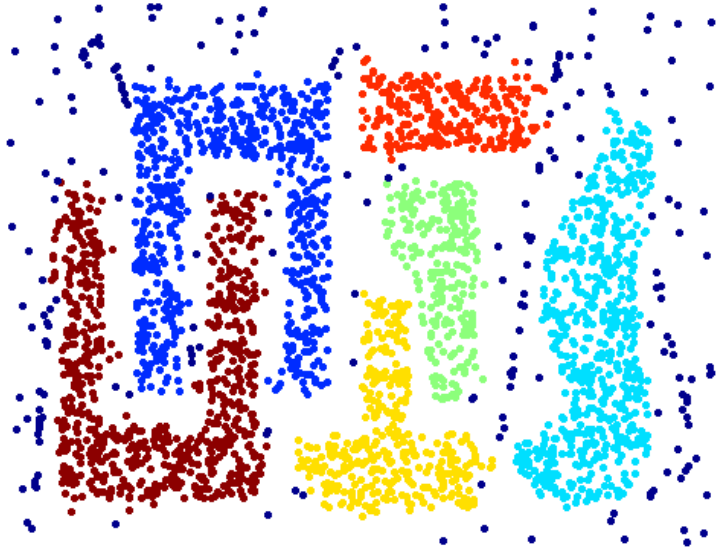
Original points (3000 points)



Point types: core, border, noise

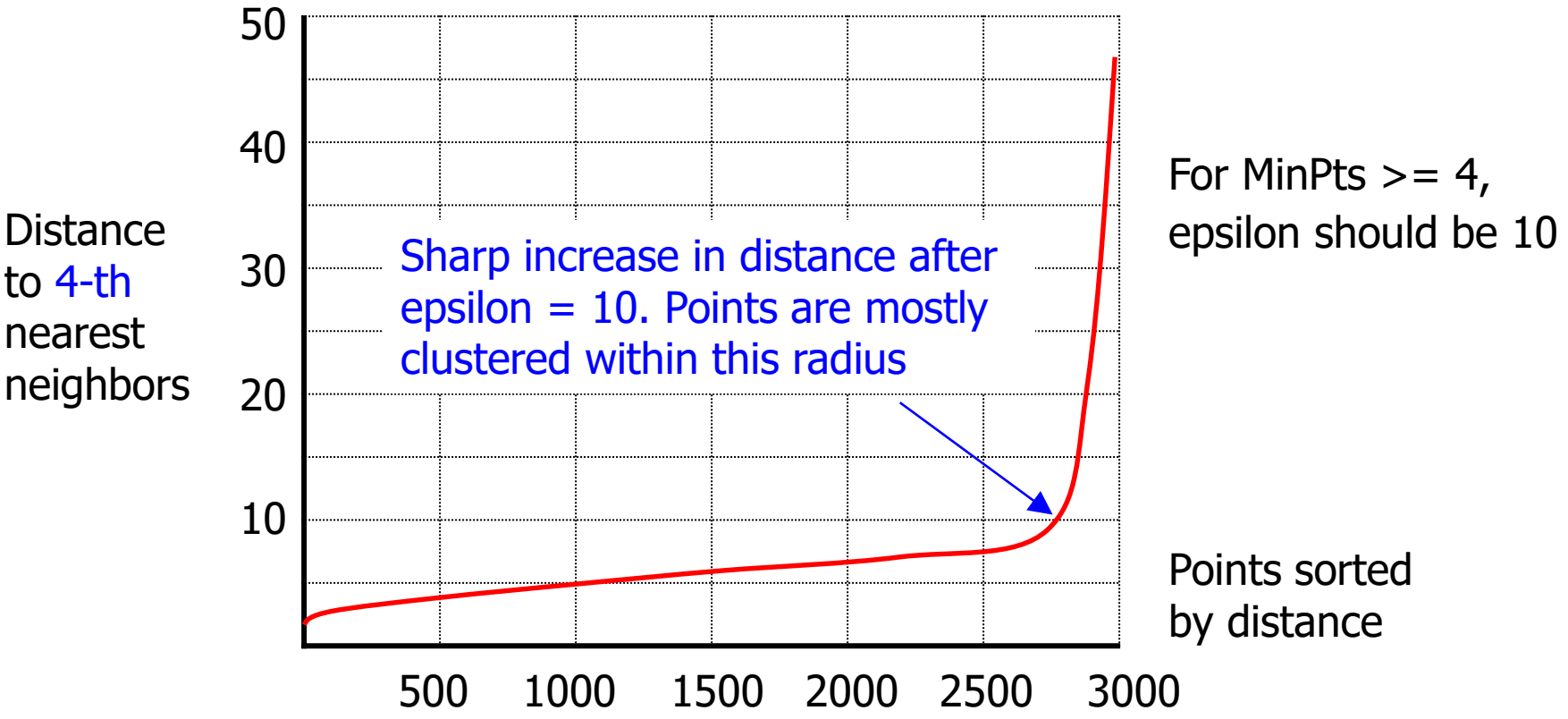
Epsilon = 10
MinPts = 4

6 clusters are automatically found



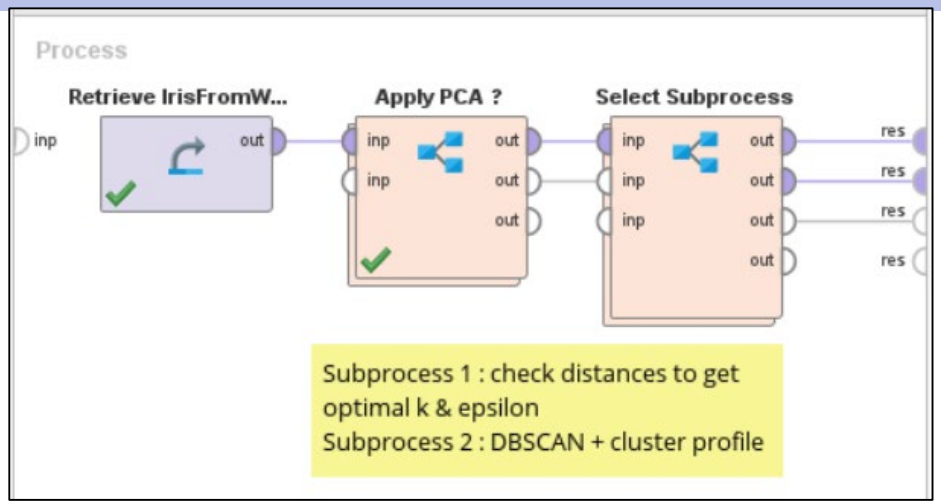
DBSCAN Parameters

- Choose k = MinPts
- Find distances to k-th nearest neighbors of all points. Sort distances and plot graph ➔ pick distance at (or slightly after) elbow point

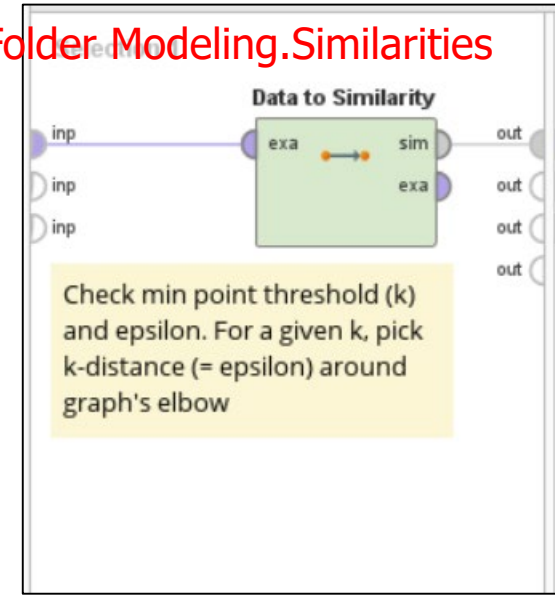


	MinPts	Epsilon
Too high	A valid point doesn't pass the threshold to be core point & it isn't border point (not in radius of other core points, or other points also fail to be core). So it is labelled as noise	Noise has big radius to cover enough points to pass MinPts. So it is labelled as core point
Too low	Noise passes the threshold easily and is labelled as core point	Radius of a valid point doesn't cover enough points (to pass MinPts) to be core and it isn't border point. So it is labelled as noise

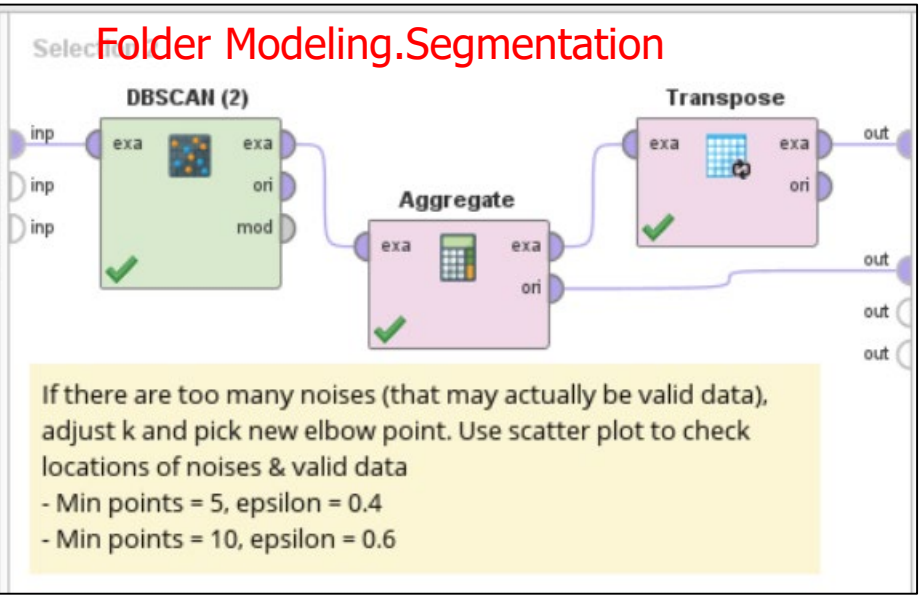
Example (3) : DBSCAN

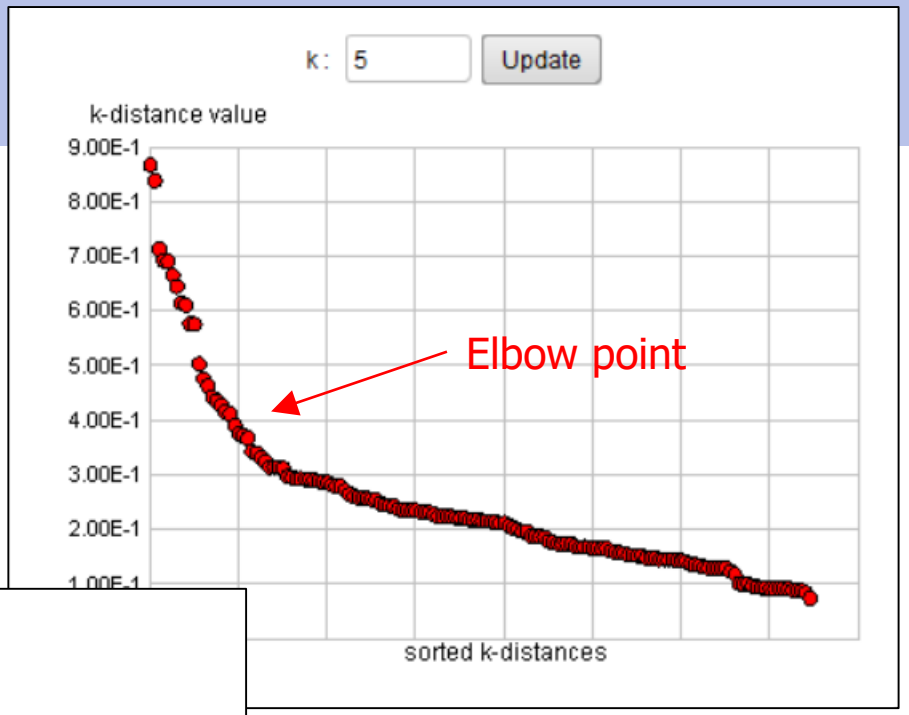


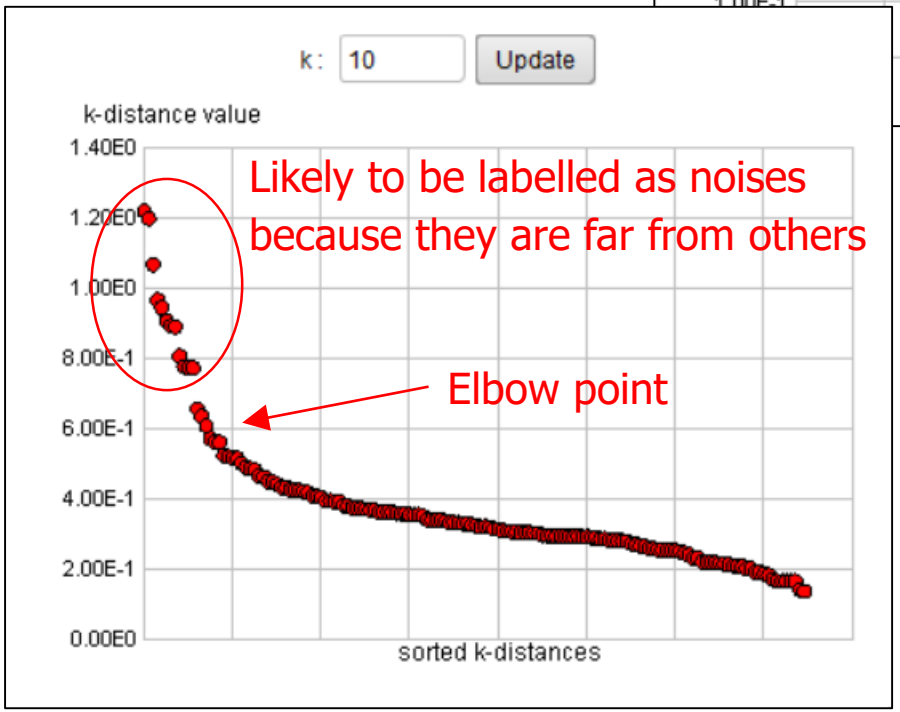
Folder Modeling.Similarities

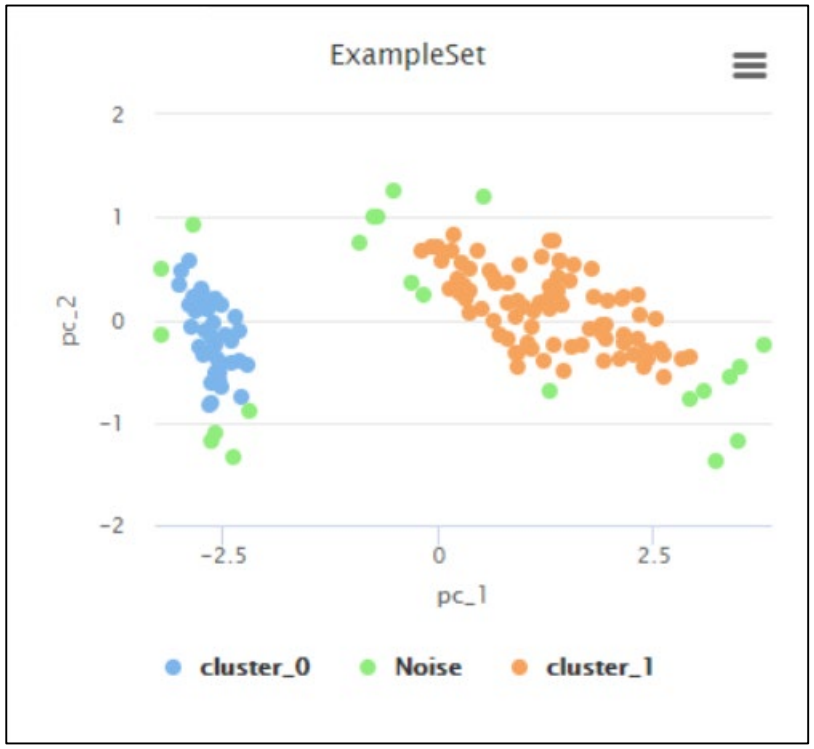


Folder Modeling.Segmentation

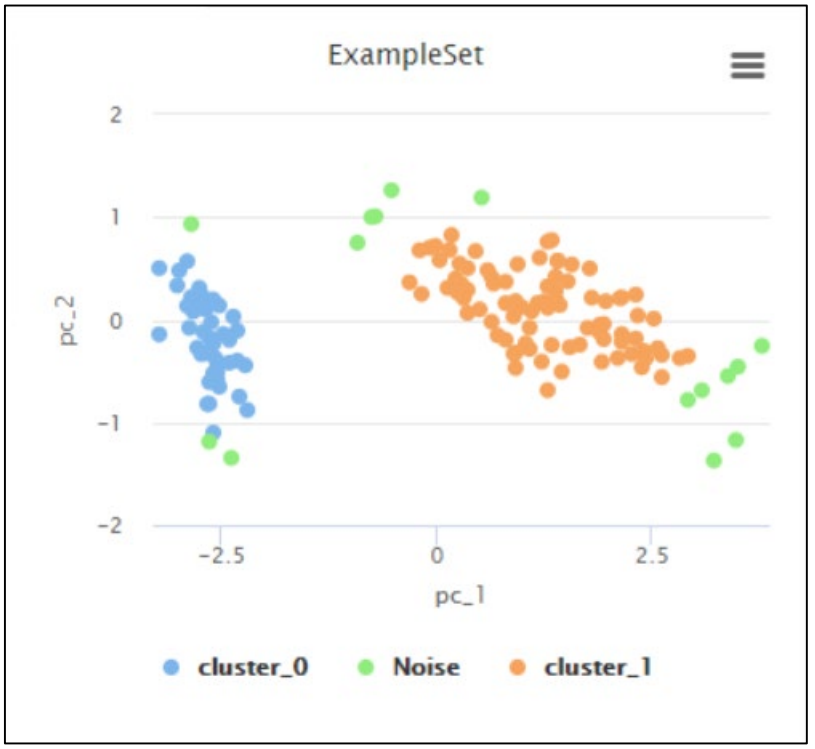



k: 5
Update
k-distance value
9.00E-1
8.00E-1
7.00E-1
6.00E-1
5.00E-1
4.00E-1
3.00E-1
2.00E-1
Elbow point
sorted k-distances


k: 10
Update
k-distance value
1.40E0
1.20E0
1.00E0
8.00E-1
6.00E-1
4.00E-1
2.00E-1
0.00E0
Likely to be labelled as noises
because they are far from others
Elbow point
sorted k-distances



Min points = 5, epsilon = 0.4
Noises = 22 points



Min points = 10, epsilon = 0.6
Noises = 15 points

Summary

- Both HAC and DBSCAN don't require #clusters from user
- HAC
  - Can cut dendrogram (flatten) to get any number of clusters
  - Dendrogram may correspond to meaningful taxonomy
  - Merging is final. This is different from K-Means and EM, where clusters are repeatedly adjusted until results are stable
- DBSCAN
  - Clusters are determined by connecting core and border points
  - Can filter out noises
  - If natural clusters have different densities, MinPts may work for one cluster, but is too high or too low for the other cluster

Clustering Tips

- Preprocessing
  - Type conversion — to use appropriate distance/similarity functions
  - Normalization — so that attributes have similar scale
  - Feature selection (but most methods require class labels which we don't have), attribute weighting (optional)
  - PCA — combine multiple attributes (optional)
- Run clustering on transformed data set to get cluster assignment
- Join tables
  - Original data set + transformed data set with cluster assignment
  - Build cluster profile from the original attributes of members (more understandable to readers)
  - Use classification models such as trees or rules to present clusters